

Factors Affecting EBPV

Initial Weights

- The ultimate solution may be affected by the initial weights of a multilayer feed-forward network.
- They are initialized at small random values.
- The choice of the initial weight determines how fast the network converges.
- The initial weights cannot be very high because the sigmoidal activation functions used here may get saturated from the beginning itself and the system may be stuck at a **local minima** or at a very flat plateau at the starting point itself.

- One method of choosing the weight w_{ij} is choosing it in the range. Better training stability and convergence.

$$\left[\frac{-3}{\sqrt{O_i}}, \frac{3}{\sqrt{O_i}} \right]$$

- where O_i is the number of processing elements j that feed-forward to processing element i .
- Consider an example where a neural network layer has 16 output neurons: Calculate the range of weight.
- Range = $\left[\left(-3 / \sqrt{16} \right) , 3 / \sqrt{16} \right] = \left[-0.75 , 0.75 \right]$

Nyugen–Widrow Initialization

- This type of initialization leads to faster convergence of network. The concept here is based on the geometric analysis of the response of hidden neurons to a single input.
- The method is used for improving the learning ability of the hidden units. The random initialization of weights connecting input neurons to the hidden neurons is obtained by the equation

$$v_{ij}(\text{new}) = \gamma \frac{v_{ij}(\text{old})}{\|\bar{v}_j(\text{old})\|}$$

where $\bar{v}_j$ is the average weight calculated for all values of i , and the scale factor $\gamma = 0.7(P)^{1/n}$ (“ n ” is the number of input neurons and “ p ” is the number of hidden neurons).

Nyugen–Widrow Initialization

- **Random Weight Initialization:** First, initialize the weights randomly in a symmetric range, typically between -0.5 and 0.5, or from a uniform distribution.
- **Normalize Weights:** The randomly initialized weights are normalized so that their magnitude is controlled. Specifically, each weight is scaled to have a norm of 1.
- **Scaling Factor:** A scaling factor is applied to the normalized weights. This factor is computed based on the number of neurons in the layer and the number of inputs. The scaling factor β is given by :

$$\beta = 0.7 \times O^{\frac{1}{n}}$$

- where: O is the number of neurons in the current layer.
 n is the number of input neurons to that layer.
- **Apply Scaling Factor:** Finally, multiply the normalized weights by this scaling factor.

$$W = \begin{bmatrix} 0.4 & -0.3 & 0.1 \\ -0.2 & 0.5 & -0.4 \\ 0.3 & -0.4 & 0.2 \\ -0.1 & 0.2 & 0.3 \end{bmatrix}$$

- Find normalization factor for each row:

$$\text{norm}(w) = \sqrt{w_1^2 + w_2^2 + \dots + w_n^2}$$

- $W1 = 0.51$
- $W2 = 0.671$
- $W3 = 0.538$
- $W4 = 0.374$

- Divide each row element with its respective normalization factor

$$\begin{bmatrix} 0.784 & -0.588 & 0.196 \\ -0.298 & 0.745 & -0.596 \\ 0.557 & -0.743 & 0.371 \\ -0.267 & 0.535 & 0.802 \end{bmatrix}$$

- Compute Scaling factor $\beta = 0.7 \times 4^{\frac{1}{3}}$

- Scale the weight matrix: $W_{\text{final}} = 1.111 \times W_{\text{norm}}$

$$\begin{bmatrix} 0.871 & -0.654 & 0.218 \\ -0.331 & 0.828 & -0.662 \\ 0.619 & -0.826 & 0.412 \\ -0.297 & 0.595 & 0.891 \end{bmatrix}$$

Learning Rate α

- The learning rate (α) affects the convergence of the BPN.
- A larger value of α may speed up the convergence but might result in overshooting, while a smaller value of α has vice-versa effect.
- The range of α from 10^{-3} to 10 has been used successfully for several back-propagation algorithmic experiments.
- Thus, a large learning rate leads to rapid learning but there is oscillation of weights, while the lower learning rate leads to slower learning.

Bias Influence

- Networks with biases can represent relationships between inputs and outputs more easily than networks without biases.
- Adding a bias to each neuron is usually desirable to offset the origin of the activation function.
- The weight of the bias is trainable similar to weight except that the input is always +1.
- The bias allows the neuron to shift the activation function left or right, enabling the neuron to activate even when the input is zero.
- Bias provides the model with additional degrees of freedom. It allows each neuron to adjust its activation independently of the input, improving the ability of the network to model complex, non-linear functions.

Momentum Factor

- The gradient descent is very slow if the learning rate α is small and oscillates widely if α is too large.
- One very efficient and commonly used method that allows a larger learning rate without oscillations is by adding a momentum factor to the normal gradient descent method.
- The use of momentum enhances the stability of the training process.
- Momentum is used to keep the training process going in the same general direction analogous to the way that momentum of a moving object behaves.
- In backpropagation with momentum, the weight change is a combination of the current gradient and the previous gradient.

Momentum Factor

- The momentum factor is denoted by $\eta \in [0, 1]$ and the value of 0.9 is often used for the momentum factor.
- This approach is more useful when some training data are very different from the majority of data.
- A momentum factor can be used with either pattern by pattern updating or batch-mode updating.
- In case of batch mode, it has the effect of complete averaging over the patterns.
- Even though the averaging is only partial in the pattern-by-pattern mode, it leaves some useful information for weight updation.

Momentum Factor

- Weight Updation Formula

$$w_{jk}(t+1) = w_{jk}(t) + \underbrace{\alpha \delta_k z_j + \eta [w_{jk}(t) - w_{jk}(t-1)]}_{\Delta w_{jk}(t+1)}$$

$$v_{ij}(t+1) = v_{ij}(t) + \underbrace{\alpha \delta_j x_i + \eta [v_{ij}(t) - v_{ij}(t-1)]}_{\Delta v_{ij}(t+1)}$$

Generalization

- The best network for generalization is BPN.
- A network is said to be generalized when it sensibly interpolates with inputs that are new to the network.
- When there are many trainable parameters for the given amount of training data, the network learns well but does not generalize well. This is usually called overfitting or overtraining.
- One solution to this problem is to monitor the error on the test set and terminate the training when the error increases.

Generalization

- With small number of trainable parameters, the network fails to learn the training data and performs very poorly on the test data.
- For improving the ability of the network to generalize from a training data set to a test data set, it is desirable to make small changes in the input space of a pattern, without changing the output components.
- This is achieved by introducing variations in the input space of training patterns as part of the training set. However, computationally, this method is very expensive.
- Also, a net with large number of nodes is capable of memorizing the training set at the cost of generalization. As a result, smaller nets are preferred than larger ones.

Number of Training Data

- The training data should be sufficient and proper.
- There exists a rule of thumb, which states that the training data should cover the entire expected input space, and while training, training-vector pairs should be selected randomly from the set.
- Assume that the input space as being linearly separable into “L” disjoint regions with their boundaries being part of hyper planes.
- Let “T” be the lower bound on the number of training patterns. Then, choosing T such that $T \gg L$ will allow the network to discriminate pattern classes using fine piecewise hyperplane partitioning.
- Also in some cases, scaling or normalization has to be done to help learning.

Number of Hidden Layer Nodes

- If there exists more than one hidden layer in a BPN, then the calculations performed for a single layer are repeated for all the layers and are summed up at the end.
- In case of all multilayer feed-forward networks, the size of a hidden layer is very important.
- The number of hidden units required for an application needs to be determined separately.
- The size of a hidden layer is usually determined experimentally.
- For a network of a reasonable size, the size of hidden nodes has to be only a relatively small fraction of the input layer.
- For example, if the network does not converge to a solution, it may need more hidden nodes. On the other hand, if the network converges, the user may try a very few hidden nodes and then settle finally on a size based on overall system performance.

Testing Algorithm for EBPN

Step 0: Initialize the weights. The weights are taken from the training algorithm.

Step 1: Perform Steps 2–4 for each input vector.

Step 2: Set the activation of input unit for x_i ($i = 1$ to n).

Step 3: Calculate the net input to hidden unit x and its output. For $j = 1$ to p ,

$$z_{inj} = v_{0j} + \sum_{i=1}^n x_i v_{ij}$$
$$z_j = f(z_{inj})$$

Step 4: Now compute the output of the output layer unit. For $k = 1$ to m ,

$$y_{ink} = w_{0k} + \sum_{j=1}^p z_j w_{jk}$$
$$y_k = f(y_{ink})$$

Use sigmoidal activation functions for calculating the output.